



TOPIC:

Movie Review

SENTIMENT ANALYZER

Presented By

HAMNA HOOR KHAN 8408
AYESHA SALEEM 8575

Visit Now

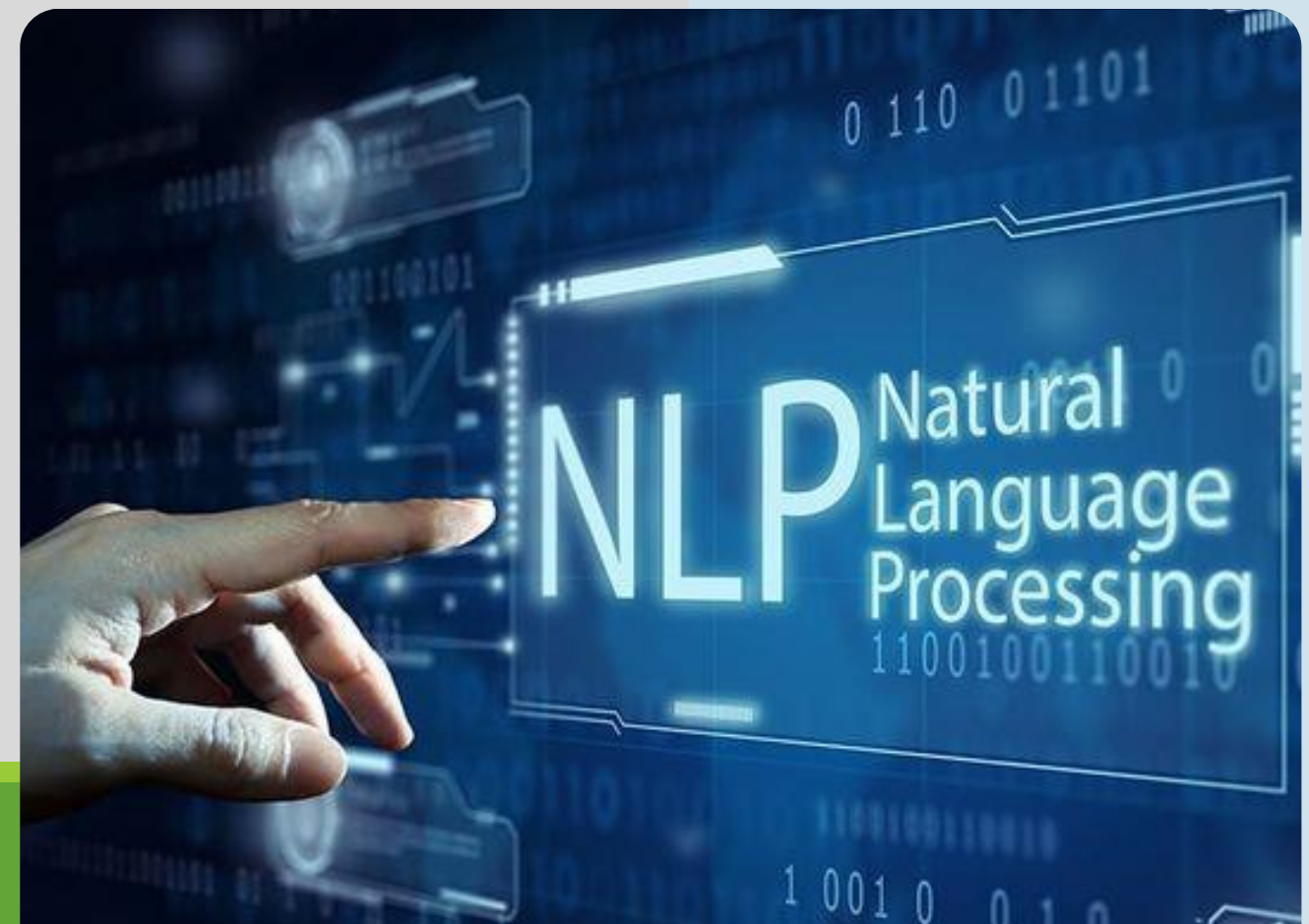
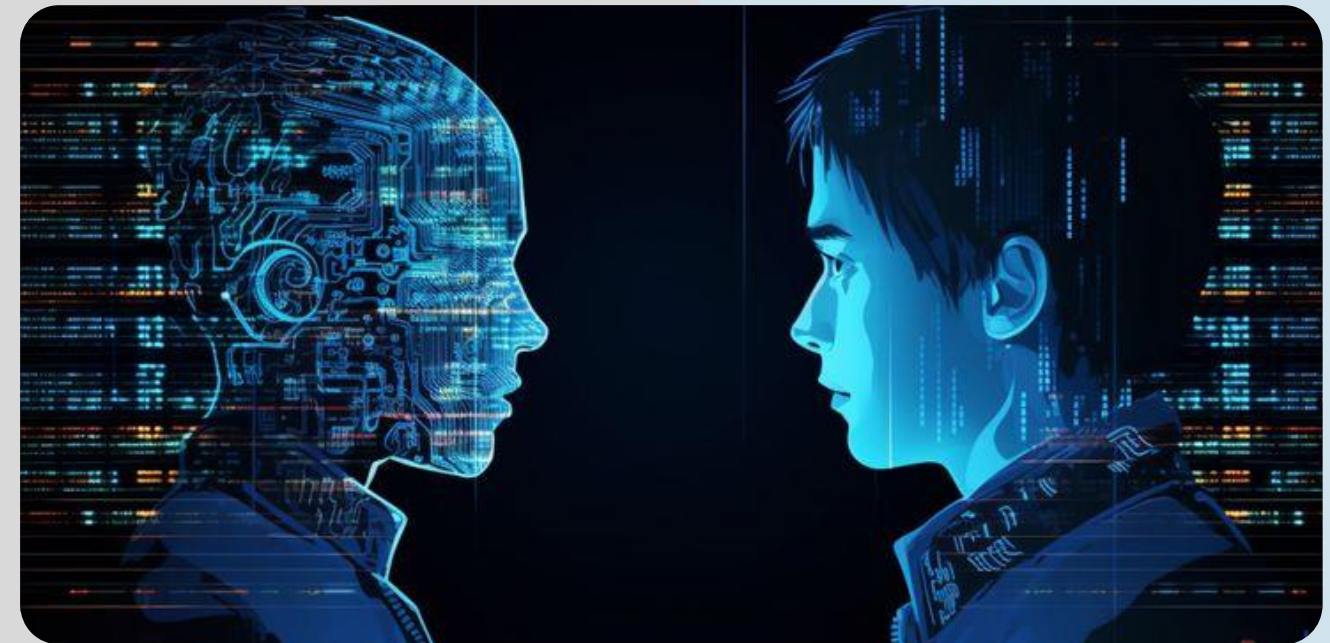


Project Goal

introduction:

The primary objective of this project is to develop a machine learning model capable of accurately classifying movie reviews as either positive or negative sentiment. This task, known as Binary Sentiment Analysis, is a fundamental application of Natural Language Processing (NLP).

[Visit Now](#)



DATA SOURCE

The model is trained and evaluated using the IMDB Dataset of 50,000 Movie Reviews, a widely used benchmark dataset for sentiment analysis. The dataset is perfectly balanced, ensuring that the model does not exhibit bias toward one class.

[See More](#)

```
# -----  
# 2. Load Kaggle Dataset  
# -----  
# File name must be: IMDB Dataset.csv  
# TODO: Replace "IMDB Dataset.csv" with the path to your CSV file in Google Drive after mounting.  
df = pd.read_csv('drive/MyDrive/movie_reviews/IMDB Dataset.csv')
```



Methodology: NLP Pipeline

- The project follows a standard machine learning pipeline for text data consisting of:
- data preparation
- feature engineering
- model training
- evaluation.

```
# .....
# 3. Text Cleaning Function
# .....
def clean_text(text):
    text = text.lower()
    text = re.sub(r"(\.)\1{2,}", r"\1", text)          # sooo → so
    text = re.sub(r"<.*?>", " ", text)                # remove HTML tags
    text = re.sub(r"https?:\/\/\S+|www\.\S+", " ", text)
    text = re.sub(r"\d+", "", text)
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = re.sub(r"\s+", " ", text).strip()
    return text

print("\nCleaning text...")
df["cleaned_review"] = df["review"].apply(clean_text)
```

CODE:

text Cleaning:
The function takes a string text and applies several common text normalization and cleaning steps:



EXPLANATION:

A	B
Code Line	Operation
text = text.lower()	Lowercase Conversion
text = re.sub(r"(\.)\1{2,}", r"\1", text)	Character Repetition Reduction
text = re.sub(r"<.*?>", " ", text)	HTML Tag Removal
`text = re.sub(r"https?:\/\/\S+	URL Removal
text = re.sub(r"\d+", "", text)	Removes all numeric digits (0-9).
text = text.translate(str.maketrans('	Punctuation Removal
text = re.sub(r"\s+", " ", text).strip()	Whitespace Normalization & Stripping
return text	Return Cleaned Text



Model Selection and Training

Model: Logistic Regression. This is a fast, linear classification algorithm that performs exceptionally well on high-dimensional, sparse data like TF-IDF vectors.

Next Slide



Training:

The model is trained on the 40,000-sample training set.

```
# -----  
# 6. Train Logistic Regression Model  
# -----  
model = LogisticRegression(max_iter=1000)  
model.fit(X_train_vec, y_train)  
  
print("\nModel Training Completed")
```

max_iter=1000:

Increased iteration limit ensures the model converges properly when dealing with a large number of features.

```
# -----  
# 7. Model Evaluation  
# -----  
y_pred = model.predict(X_test_vec)  
accuracy = accuracy_score(y_test, y_pred)  
print("\n" + "="*45)  
print("                MODEL EVALUATION")  
print("="*45)  
print(f"\nAccuracy: {accuracy:.4f}")  
  
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))
```

MODEL EVALUATION:

"The model is evaluated using accuracy and classification metrics on unseen test data."

- Model predictions are generated using test data
- Accuracy measures overall correctness of predictions
- Classification report shows:
 - Precision
 - Recall
 - F1-score
 - Support for each class

[See More](#)

```
# .....  
# 10. Predict on New Reviews  
# .....  
while True:  
    user_review = input("\nEnter a movie review (or type 'stop'): ")  
    |  
    if user_review.lower() == "stop":  
        break  
  
    cleaned = clean_text(user_review)  
    vectorized = vectorizer.transform([cleaned])  
    prediction = model.predict(vectorized)[0]  
  
    print("Sentiment Prediction:", prediction)
```

Predict on New Reviews

- Takes user input for a new movie review
- Allows continuous input until “stop” is entered
- Cleans the input text using the same preprocessing steps
- Converts text into TF-IDF features
- Uses the trained model to predict sentiment
- Displays the predicted label (positive / negative)

```
# .....  
# 9. Accuracy Graph  
# .....  
plt.figure()  
plt.bar(["Logistic Regression Accuracy"], [accuracy])  
plt.ylim(0, 1)  
plt.title("Model Accuracy on IMDB Dataset")  
plt.ylabel("Accuracy")  
plt.show()
```

ACCURACY GRAPH

- Visual representation of model performance
- Bar graph shows Logistic Regression accuracy
- Y-axis ranges from 0 to 1 (valid accuracy range)
- Makes performance easy to interpret
- Helpful for quick comparison with other models



OUTPUT:

**Regulatory
and ethical
challenges
include
liability
frameworks
and
algorithmic**

```
Dataset Loaded Successfully
Total Reviews: 50000

4954
22212 This is without doubt the most exciting and satisfying film
48545
40684
45253

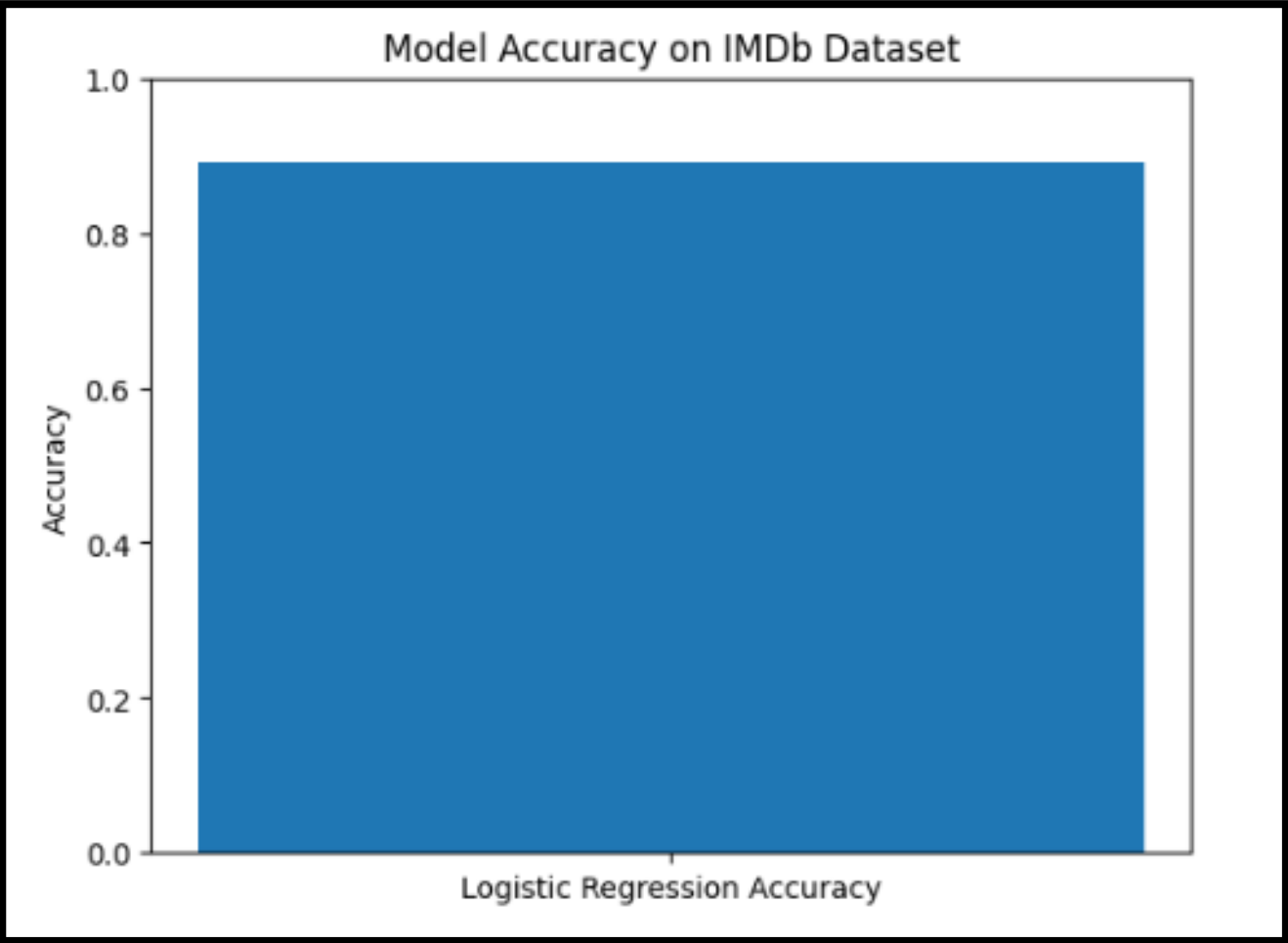
      sentiment
4954    positive
22212    positive
48545    negative
40684    negative
45253    negative

Cleaning text...
Training samples: 40000
Testing samples: 10000

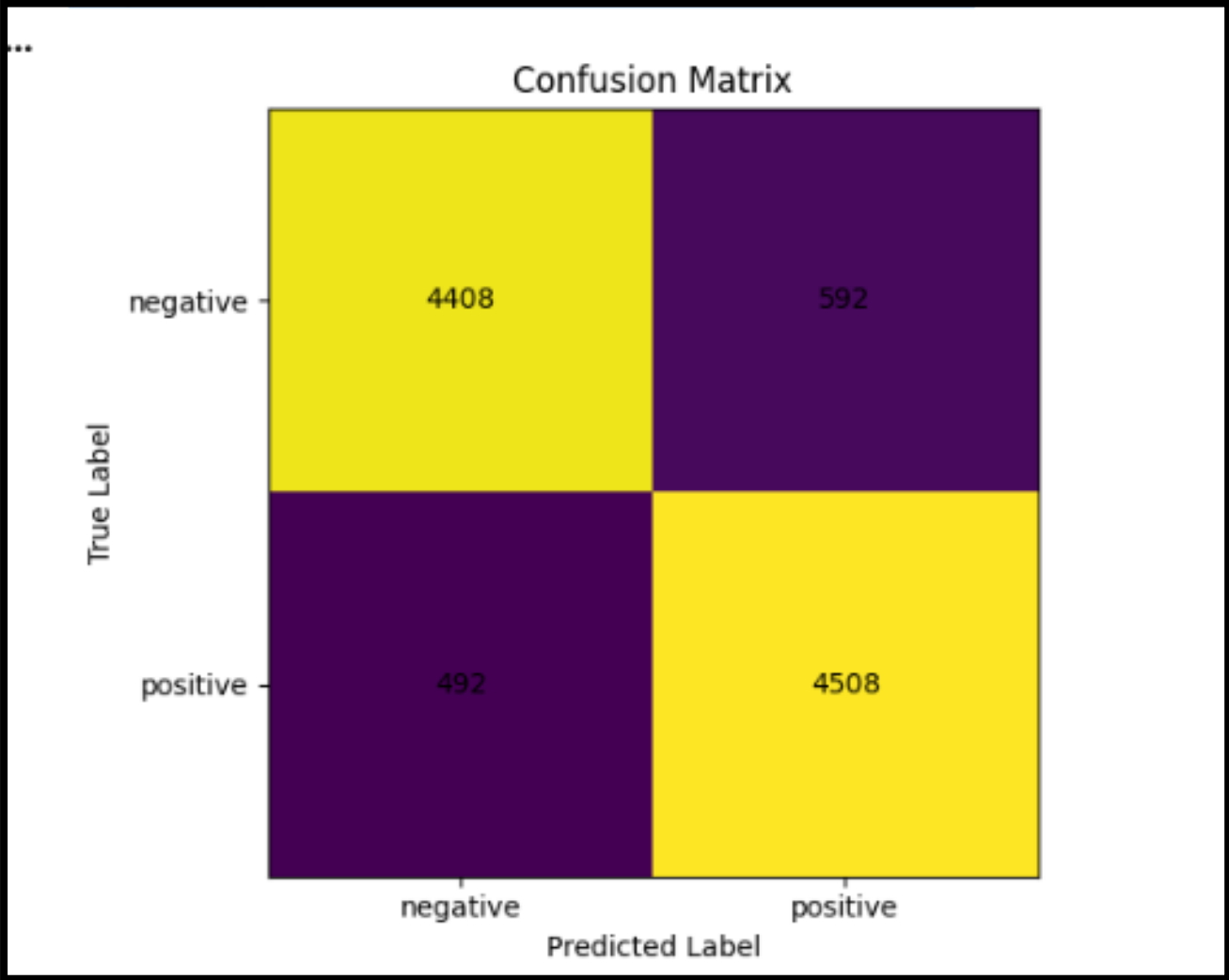
Vectorization Complete
Feature matrix shape: (40000, 5000)
```



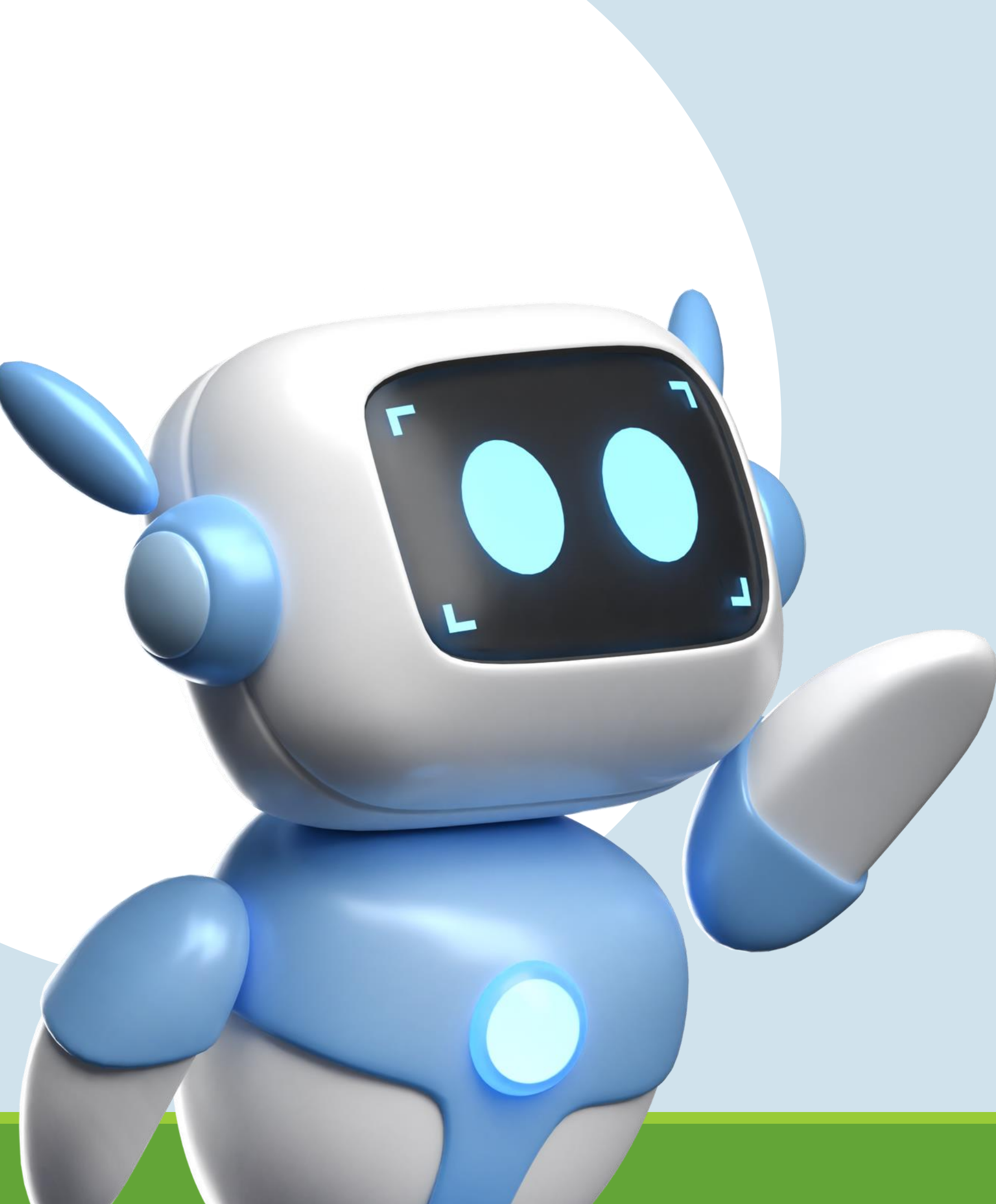
Key developments include:



Reusable Rockets
In-Situ Resource
Utilization: Lunar/Mars
resources



Reusable Rockets
In-Situ Resource Utilization:
Lunar/Mars resources



THANK YOU

See You Next Time

